

ALGORITMOS Y ESTRUCTURAS DE DATOS

Facultad de Ingeniería — UNER

Trabajo Práctico N° 1

Integrantes:

Spiga Edgardo
Mickol Reisenstadt

Fecha de entrega: 04/05/2025

Repositorio: <https://github.com/MickRst/AYED2026C1-REISENSTADT-SPIGA>

1. Introducción

Este trabajo práctico aborda tres problemas vinculados al diseño e implementación de estructuras de datos y algoritmos. En el Problema 1 se implementa el TAD Lista Doblemente Enlazada, una estructura dinámica donde cada nodo mantiene referencias al nodo anterior y siguiente, lo que permite operaciones eficientes en ambos extremos. En el Problema 2 se modela el juego de cartas 'Guerra' usando esa misma estructura como base del mazo. En el Problema 3 se implementan y comparan tres algoritmos de ordenamiento: burbuja, quicksort y radix sort.

Los conceptos teóricos aplicados incluyen tipos abstractos de datos (TAD), manejo de excepciones, complejidad algorítmica en notación O grande, programación orientada a objetos con decoradores `@property`, docstrings con precondiciones y postcondiciones, y pruebas unitarias con `unittest`.

2. Desarrollo de los problemas

2.1. Problema N° 1: Lista Doblemente Enlazada

A. Análisis y planteo

El problema requiere una estructura donde cada nodo conozca a su predecesor y sucesor. Las operaciones críticas son inserción y extracción en los extremos en $O(1)$, recorrido completo en $O(n)$, y copia sin usar estructuras auxiliares de Python. Se eligió la lista doblemente enlazada porque mantener referencias directas a cabeza y cola evita recorrer toda la estructura para operar en los extremos.

B. Implementación

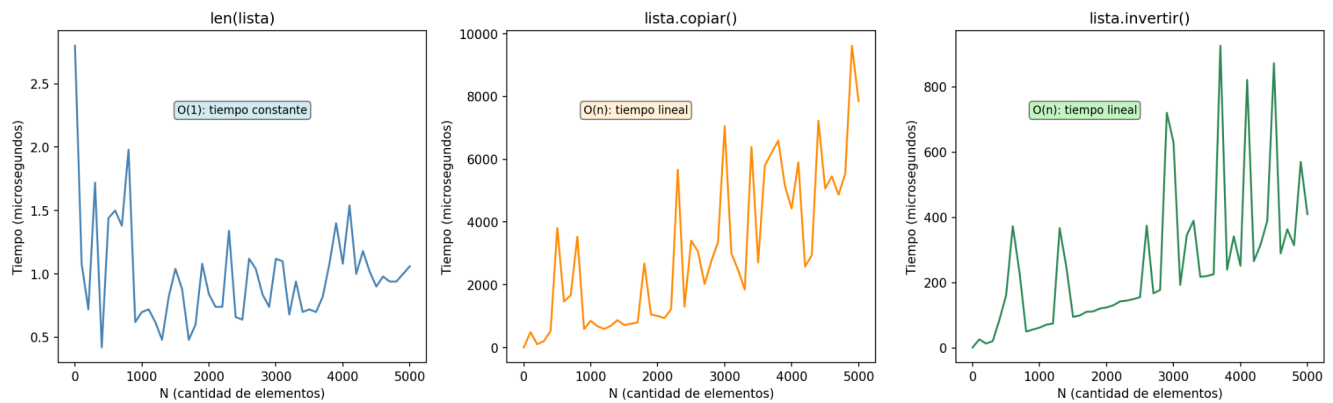
Se implementaron las clases `_Nodo` y `ListaDobleEnlazada` en biblioteca `ayed_fiuner/ayedfiuner/estructuras/LDE.py`. Los atributos privados `_cabeza`, `_cola` y `_tamanio` se exponen mediante `@property`. Los métodos principales y sus complejidades son:

- `agregar_al_inicio / agregar_al_final`: $O(1)$, actualizan cabeza o cola directamente.
- `insertar(item, posicion)`: $O(1)$ en extremos, $O(n)$ en caso general.
- `extraer(posicion)`: $O(1)$ en extremos (soporta índice -1), $O(n)$ en caso general.
- `copiar()`: $O(n)$, recorre la lista una vez usando `agregar_al_final`.
- `invertir()`: $O(n)$, intercambia punteros anterior/siguiente en cada nodo.
- `concatenar(lista)`: $O(n)$, recorre la lista argumento y agrega al final.
- `__len__`: $O(1)$, devuelve directamente el atributo `_tamanio`.
- `__add__`: usa `copiar()` y `concatenar()` para no repetir lógica.
- `__iter__`: generador que recorre los nodos devolviendo el dato de cada uno.

C. Resultados y pruebas

La clase pasó los 14 tests provistos por la cátedra. Las gráficas de tiempos confirman los órdenes de complejidad esperados: `len()` muestra una línea prácticamente plana ($O(1)$), mientras que `copiar()` e `invertir()` crecen linealmente con N ($O(n)$). Las variaciones puntuales se deben a interrupciones normales del sistema operativo durante la medición.

ListaDobleEnlazada: N vs Tiempo de ejecución



2.2. Problema N° 2: Juego de Cartas Guerra

A. Análisis y planteo

El juego requiere una estructura que permita sacar cartas por arriba y agregar por abajo al ganar un turno, lo que corresponde a una cola (FIFO). Esto se implementa eficientemente con la LDE donde la cabeza representa la parte de arriba del mazo y la cola la parte de abajo, garantizando $O(1)$ en todas las operaciones del mazo.

B. Implementación

Se implementó la clase Mazo en `proyecto_2/modules/mazo.py` como un envoltorio de ListaDobleEnlazada con tres métodos principales:

- `poner_carta_arriba(carta)`: llama a `agregar_al_inicio()` — $O(1)$.
- `poner_carta_abajo(carta)`: llama a `agregar_al_final()` — $O(1)$.
- `sacar_carta_arriba(mostrar=False)`: llama a `extraer(0)`, opcionalmente hace la carta visible. Lanza `DequeEmptyError` si el mazo está vacío — $O(1)$.

C. Resultados y pruebas

Se pasaron los 5 tests de la cátedra: 2 de la clase Mazo y 3 del juego completo, verificando partidas con ganador 1, ganador 2 y empate con seeds específicas.

2.3. Problema N° 3: Algoritmos de Ordenamiento

A. Análisis y planteo

Se implementaron tres algoritmos con complejidades distintas para compararlos empíricamente. Burbuja es simple pero costoso ($O(n^2)$). Quicksort aplica divide y vencerás obteniendo $O(n \log n)$ en promedio. Radix sort ordena dígito a dígito sin comparar elementos entre sí, siendo prácticamente lineal $O(n \cdot k)$ para números de tamaño fijo.

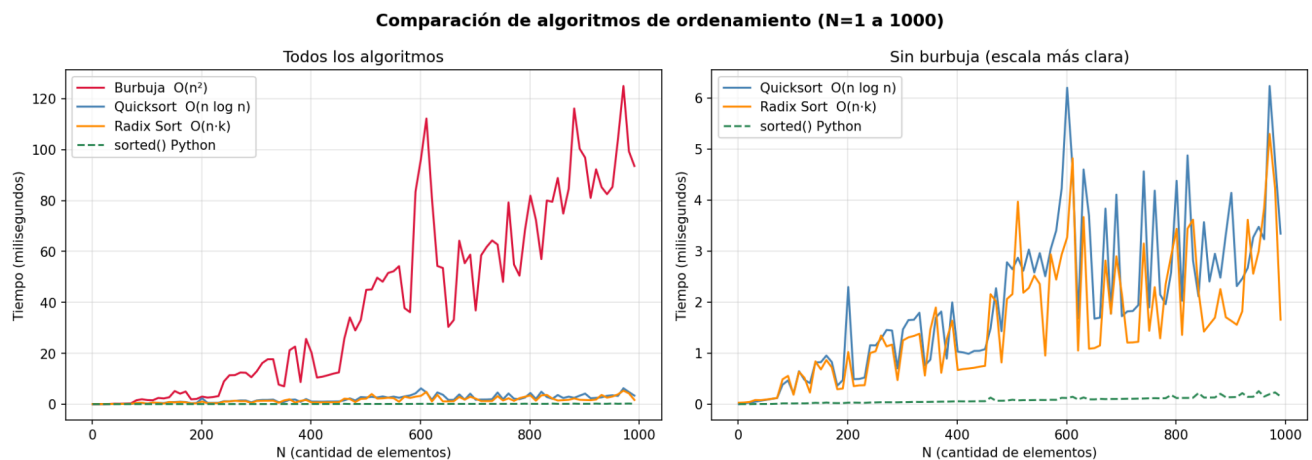
B. Implementación

Los tres algoritmos se implementaron en biblioteca_ayed_fiuner/ayedfiuner/algoritmos/ordenamiento.py. Todos devuelven una lista nueva sin modificar la original:

- burbuja(lista): dos bucles anidados, compara pares adyacentes e intercambia si están en orden incorrecto. $O(n^2)$ siempre.
- quicksort(lista): elige el elemento central como pivote, separa en menores/iguales/mayores y aplica recursión. $O(n \log n)$ promedio.
- radix_sort(lista): aplica counting sort estable por cada posición de dígito. $O(n \cdot k)$ donde $k=5$ para números de 5 dígitos.

C. Resultados y pruebas

Se verificó correctitud comparando con sorted() sobre 500 números aleatorios de 5 dígitos. La gráfica muestra cómo burbuja crece de forma parabólica mientras quicksort y radix sort permanecen muy por debajo. sorted() de Python es el más rápido porque usa Timsort implementado en C, con $O(n \log n)$ garantizado.



3. Análisis de resultados específicos / Discusión

La Lista Doblemente Enlazada resultó ser la estructura adecuada para ambos problemas. Para el Mazo del juego Guerra, las operaciones críticas son sacar de arriba y poner abajo, ambas $O(1)$ gracias a las referencias directas a cabeza y cola.

En los algoritmos de ordenamiento, la diferencia entre $O(n^2)$ y $O(n \log n)$ se hace evidente ya a partir de $N=500$: burbuja llega a los 100 ms mientras quicksort y radix sort permanecen por debajo de 5 ms. Esto ilustra por qué la elección del algoritmo impacta más que la velocidad del hardware para entradas de tamaño mediano o grande.

La función sorted() de Python usa Timsort, un algoritmo híbrido que combina merge sort para listas grandes e insertion sort para sublistas pequeñas ya parcialmente ordenadas. Su ventaja práctica es que está implementado en C y aprovecha el orden parcial existente en datos reales.

4. Declaración de Uso Ético de Inteligencia Artificial

Herramientas utilizadas: Gemini (Google)

Descripción del uso:

- Asistencia en código: Se utilizó para identificar errores durante la ejecución, en particular un `ModuleNotFoundError` relacionado con `matplotlib` y problemas con las rutas al ejecutar los scripts de generación de gráficas en Windows.
- Consulta teórica: Se consultó para repasar conceptos de complejidad algorítmica y el funcionamiento interno de los algoritmos de ordenamiento.

Declaración de responsabilidad:

Los integrantes del grupo declaramos que hemos revisado y validado personalmente toda la información y código sugerido por las herramientas de IA. Entendemos los conceptos presentados en este informe y asumimos la responsabilidad académica por la integridad y veracidad del contenido. El trabajo final es producto de nuestro propio análisis y criterio.

5. Trabajo en equipo

Estimación inicial: Al comenzar calculamos que el TP nos iba a llevar unas 15 horas en total.

Organización y roles:

Ambos integrantes participamos en todas las actividades del TP. Nos juntamos en la casa de Edgardo Spiga, donde usamos su computadora para escribir y ejecutar los códigos. Mickol Reisenstadt se encargó de revisar los materiales teóricos y los apuntes de cátedra mientras avanzábamos con la implementación, y también tomó la iniciativa en la redacción del informe. Las decisiones de implementación se tomaron en conjunto.

Tiempo real y diferencias: El tiempo real fue muchísimo mayor al estimado. La principal dificultad fue que varios contenidos básicos necesarios para el TP, no los habíamos visto en profundidad en FuPro, lo que requirió tiempo adicional para entenderlos antes de poder implementarlos. También surgieron problemas técnicos al configurar el entorno en Windows que no habíamos contemplado en la estimación inicial.

6. Conclusiones

Se cumplieron todos los objetivos del trabajo: la Lista Doblemente Enlazada pasó los 14 tests de la cátedra, el juego Guerra pasó los 5 tests correspondientes, y los tres algoritmos de ordenamiento funcionaron correctamente y fueron comparados tanto de forma teórica como empírica.

La mayor dificultad fue manejar correctamente los punteros en la LDE, especialmente en los casos borde como lista vacía o con un solo elemento. Los tests unitarios fueron fundamentales para detectar y corregir estos errores.

Lo más valioso del trabajo fue ver en la práctica cómo la elección de una estructura de datos o algoritmo impacta directamente en el rendimiento. Ver la diferencia entre burbuja y quicksort en la gráfica hace tangible algo que de otra forma parece solo teoría.

7. Referencias y Bibliografía

- Miller, B., & Ranum, D. (2013). Solución de problemas con algoritmos y estructuras de datos usando Python (M. Orozco-Alzate, Trad.; 2.a ed.).
- Documentación oficial de Python — docs.python.org
- Material de cátedra — Algoritmos y Estructuras de Datos, UNER 2026
- Repositorio del TP — <https://github.com/MickRst/AYED2026C1-REISENSTADT-SPIGA>